ML Subtree-Pruning-and-Regrafting (SPR) for IQ-TREE 3

Detailed implementation roadmap for the local IQ-TREE 3 fork — 21 July 2026

1. Executive summary

This project is feasible. IQ-TREE 3 already contains a dormant, experimental maximum-likelihood SPR implementation in `tree/phylotree.cpp`, wired to a `-spr` switch. It provides useful evidence about the intended topology representation, but it is not a safe foundation for a production search without refactoring: it has hard-coded search limits, debug output, incomplete state handling, mixed legacy/new routines, and does not participate in the mature `IQTree` search lifecycle.

The recommended outcome is a bounded-radius ML-SPR refinement/search mode built around an explicit transactional move API. It should generate legal prune/regraft candidates without changing the tree, evaluate candidates through reversible topology changes, fully re-evaluate the strongest candidates, accept at most one move per round, then update likelihood, branch lengths, checkpoints, candidate-tree state, and diagnostics exactly as the production NNI search does.

> **Recommended first release.** Support unrooted, binary, single-tree likelihood analyses; bounded-radius SPR; serial candidate evaluation; one accepted move per round; and full branch reoptimization after acceptance. Add partitioned/supertree and specialized tree types only as explicit follow-on work.

2. Goals, non-goals, and success criteria

2.1 Goals

- Provide an optional ML-SPR topology search/refinement facility that is correct, reproducible, and measurable.
- Use IQ-TREE's own tree, neighbor, likelihood, optimization, constraint, and checkpoint infrastructure.
- Guarantee that a rejected candidate leaves the topology, branch lengths, caches, and score unchanged.
- Guarantee that every accepted move improves a freshly computed likelihood by more than the configured epsilon.
- Expose bounded computational cost through radius, candidate-cap, round-cap, and time controls.
- Leave existing NNI-only behavior identical when SPR is disabled.

2.2 Non-goals for the first release

- Unbounded global SPR enumeration on large trees.
- Batch application of multiple allegedly non-conflicting SPR moves.
- Parallel candidate scoring before serial correctness and cache discipline are demonstrated.
- Automatic support for every specialized tree subclass.
- Replacing IQ-TREE's existing stochastic NNI/IQP search by default.

2.3 Definition of done

1. Core supported inputs complete a bounded SPR run without topology or cache corruption.

2. For every accepted move, independently recomputing likelihood after the move verifies a gain greater than epsilon.
3. For every rejected candidate, apply/rollback restores the exact Newick topology and branch-length vector.
4. Existing regression tests pass unchanged with SPR disabled.
5. New tests cover legal/illegal moves, candidate generation, rollback, score monotonicity, constraints, resume, and deterministic output.
6. Report files and terminal output state parameters, candidate counts, accepted moves, likelihood gain, and timing.

3. Current repository map and implications

Area	Files	Role in this project
Existing SPR prototype	`tree/phylotree.h`, `tree/phylotree.cpp`	Contains `SPRMove`, `SPRMoves`, `optimizeSPR*`, candidate assessment, and commented prune/regraft helpers. Use as a behavioral reference only.
Production topology search	`tree/iqtree.h`, `tree/iqtree.cpp`	Owns `doTreeSearch`, NNI evaluation, move application, score updates, candidate trees, perturbation, and search accounting. This should own the SPR search orchestrator.
Base tree / likelihood	`tree/phylotree.*`, `tree/phylonode.*`, `tree/node.*`	Owns tree representation, neighbor links, partial likelihoods, branch optimization, and low-level topology mutation.
Workflow entry point	`main/phyloanalysis.cpp`	Currently branches to the prototype when `params.tree_spr` is true. Replace with an integrated search/refinement call.
Options	`utils/tools.h`, `utils/tools.cpp`	Add validated public parameters and defaults; retain `-spr` as an alias only if backward compatibility is wanted.
Constraints	`tree/constrainttree.*`	NNI has compatibility checks. SPR needs an equivalent topology-level compatibility gate before a candidate is evaluated or accepted.
Derived tree types	`tree/phylosupertree*`, `tree/phylotreemixlen*`, `tree/iqtreemix*`	Must be explicitly supported via overrides or rejected with a clear error; inherited pointer rewiring is unsafe by assumption.
PLL reference	`pll/searchAlgo.c`	Contains separate SPR enumeration and rollback patterns. Study it; do not directly combine representations.

> **Important finding.** The legacy top-level method sets radius to 10, limits stored moves to 20, loops up to 100 rounds, and invokes the legacy traversal. These values must become parameters and must not define algorithmic correctness.

4. Architecture and move semantics

4.1 Tree model assumptions for version 1

- Tree is connected, unrooted, and strictly binary internally.
- An SPR move cuts one directed parent/subtree edge. Suppressing its former internal attachment point joins its two remaining incident branches.
- Regrafting subdivides a target edge, reconnects the suppressed internal node, and attaches the pruned subtree.
- The target edge must be outside the pruned subtree. Moves that recreate the same topology are excluded.
- Branch lengths may be initialized from split/merged local lengths during screening, then optimized before final acceptance.

4.2 Move-state contract

Replace the current pointer-only candidate structure with a value-like description plus a separate rollback state. A candidate must be valid before any mutation and should identify edges by stable endpoint/node IDs or safely scoped pointers.

```
SPRMove
  prune_child, prune_parent       // directed cut edge
  regraft_a, regraft_b            // target undirected edge
  radius, screening_score, exact_score
  candidate_id / generation round

SPRRollback
  original neighbor endpoints and lengths for all rewired edges
  original root/direction state if applicable
  changed-cache frontier or a conservative "clear all" marker
  optional branch-length snapshot for exact candidate evaluation
```

Do not place transient neighbor iterators in a stored candidate unless mutation is impossible between generation and use. Iterators and pointers are appropriate only inside a single apply/evaluate/rollback transaction.

4.3 Low-level public/private API

API	Responsibility	Required invariant
`isLegalSPR(move)`	Tests membership, non-identity, binary constraints, root conditions, and constraints.	Never mutates the tree.
`applySPR(move, rollback)`	Rewires exactly one legal move and records all state required to undo it.	Produces a connected binary tree.
`rollbackSPR(rollback)`	Restores pre-apply topology and branch lengths.	Restores exact prior representation.
`invalidateSPRPartials(...)`	Invalidates all likelihood data affected by topology or branch-length changes.	No stale partial is used after mutation.
`scoreSPR(move, mode)`	Applies, optimizes, scores, and rolls back unless explicitly asked to commit.	Leaves no mutation on a rejected/evaluated candidate.
`commitSPR(move)`	Applies final move and optimization, updates score/state.	New `curScore` is exact.

5. Detailed phased implementation plan

Phase 0 — Baseline, fixtures, and instrumentation

Purpose: make later failures observable and distinguish algorithmic problems from existing build/test issues.

1. Build the exact fork and run the existing automated tests.
2. Identify the project test conventions under `test_scripts` and add a dedicated SPR fixture directory without changing unrelated tests.
3. Create small fixtures: 4–8 taxon trees for topology legality; 8–20 taxon alignments where NNI gets stuck but a known SPR move improves likelihood; and no-improvement controls.
4. Add a test helper that extracts canonical topology/Newick, edge count, taxon set, and branch-length vector before/after an operation.
5. Add a debug-only tree validator: connected traversal visits every node once; each neighbor link is reciprocal; no degree-two internal node remains after an applied move; leaf names/IDs are unchanged; expected edge count holds.
6. Run the existing `-spr` option on those fixtures and document results; do not use it as an oracle.

Exit criteria: baseline tests pass, fixtures are committed, and a failed rollback can be diagnosed from preserved topology/score output.

Phase 1 — Configuration and explicit algorithm policy

Purpose: prevent hard-coded experimental behavior from leaking into the new design.

1. Add structured parameters to `Params` in `utils/tools.h`: enabled/mode, radius, top-K exact candidates, max rounds, likelihood epsilon, local optimization policy, and optional time budget.
2. Initialize defaults in `utils/tools.cpp`. Recommended conservative defaults: ref[illegible]mode; radius 3–5; top K 10–20; one accepted move/round; epsilon equal to the tree-search tolerance; a finite round limit.

3. Parse documented long options, validate ranges, and reject incompatible combinations. Preserve `-spr` as a compatibility alias mapped to `--spr-mode refine`, or deprecate it with a warning.
4. Define search modes precisely: `refine` invokes SPR after existing tree search; `rch` may invoke it at configured points in the stochastic loop; do not expose a mode before it is implemented.
5. Add a configuration report block showing all effective values and a deterministic random seed if random candidate sampling is later added.

Exit criteria: no topology code reads hidden hard-coded radius/candidate/round limits.

Phase 2 — Transactional topology mutation

Purpose: make all SPR operations reversible and independently testable before likelihood is involved.

1. Read the existing neighbor-update methods and define one canonical way to replace each endpoint. Avoid ad hoc vector edits in multiple call sites.
2. Implement `applySPR` to: validat ndpoints; save original reciprocal neighbor objects and lengths; suppress the prune parent; split target edge; and reconnect the pruned subtree.
3. Implement `rollbackSPR` only from record state; it must not infer old topology from the currently mutated tree.
4. For every changed edge, preserve both directional branch lengths. Ensure split lengths are explicit parameters rather than hidden half-edge assumptions.
5. Handle root conventions deliberately. For version 1, reject rooted/nonreversible conditions if a valid generic update cannot be demonstrated.
6. Add tests over every legal move of small trees: apply; validate; rollback; validate; compare topology and all lengths to snapshots.

Exit criteria: thousands of randomized move/rollback iterations pass under assertions and sanitizers where available.

Phase 3 — Candidate generation

Purpose: enumerate valid bounded SPR candidates without accidental topology changes.

1. Define a directed prune edge enumeration. Exclude leaf/root special cases only where mathematically or structurally necessary; pruning a leaf is normally a valid SPR case.
2. For each directed prune edge, mark or traverse the pruned component so target edges within it are excluded.
3. Run a breadth-first traversal from the former attachment frontier to find eligible target edges within `spr_radius`. Count distance consistently and document it.
4. Exclude the original attachment edge and moves that yield the same unrooted split set.
5. Deduplicate candidates. Canonicalize a move so traversing from the opposite direction cannot create a duplicate.
6. Apply constraint compatibility before expensive likelihood work. Initial safe strategy: temporarily apply then invoke an existing tree-level constraint check, roll back, and cache the boolean. Later replace with a split-level precheck if needed.
7. Add candidate count/topology uniqueness tests on known 5–10 taxon trees.

Exit criteria: candidate enumeration is deterministic; all candidates are legal and unique; no candidates leak across the pruned component.

Phase 4 — Likelihood-cache and branch-length correctness

Purpose: score a candidate safely.

1. Start with the conservative correctness implementation: after every temporary move, call the existing full partial-likelihood invalidation routine, optimize selected local branches, calculate likelihood, then roll back and invalidate again.
2. Verify score symmetry: original score before evaluation equals original score after rollback within numeric tolerance.
3. Define screening optimization. Initial option: optimize the new attachment branch and the split target edges, then calculate likelihood. Do not claim exactness for this score.
4. Define exact optimization. Apply a shortlist candidate, invoke existing branch optimization sufficiently broadly (initially all branches), then compute likelihood from the normal exact path.
5. Record branch lengths from temporary exact evaluation only inside rollback state. Do not retain cache pointers or partial-likelihood arrays in move records.
6. Once correct, profile cache clearing. Optimize only affected partials only when equivalence to full clearing is proven with tests.

Exit criteria: full recomputation agrees with every exact candidate score; rollback score agrees with pre-move score.

Phase 5 — Selection and search loop

Purpose: turn candidates into a monotone local-search algorithm.

1. Generate candidates from the current committed tree.
2. Score candidates using the screening policy; retain a bounded priority list of the best K.
3. Exactly evaluate shortlist candidates, always rolling back after evaluation.
4. Select the best candidate with exact score greater than `curScore + epsilon`.
5. Apply and commit that one candidate; optimize required branches; calculate the exact final score; assert final score is at least the selected score minus permitted numerical tolerance.
6. Update `curScore`, best score/tree state, and instrumentation.
7. Repeat until no improving move, maximum rounds, time budget, or user interruption.
8. Initially do not apply multiple candidates in a round. SPR neighborhoods overlap broadly, so NNI-style compatibility rules are insufficient without more work.

Exit criteria: every committed round is strictly likelihood-improving and final output is a local optimum within the configured bounded SPR neighborhood.

Phase 6 — IQTree integration

Purpose: make SPR a first-class participant in the application workflow rather than a side call.

1. Introduce a virtual/overridable orchestration API in `IQTree`, such as `doSPRSearch(const SPRSearchConfig&)`, while placing low-level mutation in `PhyloTree`.
2. Follow the NNI search's ownership rules for `curScore`, best-tree status, candidate sets, model/branch optimization, and output statistics.
3. Update `main/phyloanalysis.cpp` to invoke the integrated API instead of calling the prototype directly.
4. Decide exactly where refinement runs: recommended first release is after the normal tree search completes and before final reporting/checkpoint output.
5. Ensure candidate-tree/checkpoint serialization sees only committed trees. Temporary scored candidates must never leak into resumable state.
6. Verify interruption and checkpoint timing: checkpoint only at round boundaries after a committed state.
7. Set and report final score through the same paths used by NNI; eliminate comparisons against stale `getCurScore()`.

Exit criteria: an SPR-refined run has correct final score, candidate-tree state, checkpoint/resume behavior, and report output.

Phase 7 — Derived tree types and compatibility gates

Purpose: avoid silent corruption for representations whose topology changes must be propagated to linked structures.

Variant	Initial release policy	Follow-on work
Ordinary `IQTree` / `PhyloTree`	Supported.	Performance improvements.
Partitioned/supertree	Reject with a direct message unless a dedicated override is implemented.	Mirror the propagation/linked-neighbor logic used by `PhyloSuperTree::doNNI`; validate every partition.
Unlinked partitions	Reject or run independently only if that is a documented user-level meaning.	Implement per-partition orchestration and score aggregation.
Mixture branch lengths	Reject initially.	Define per-mixture branch-length snapshots and optimization.
Rooted/nonreversible	Reject initially unless root-direction recomputation is proven.	Implement and test directional-cache/root updates.
Constraints	Support only after legality check is tested.	Optimize prechecks based on split compatibility.

Exit criteria: every unsupported mode fails early and clearly; no inherited base-tree rewiring is used accidentally.

Phase 8 — User interface, diagnostics, and documentation

1. Add command reference and user-guide text: method, scope, cost, defaults, limitations, reproducibility, and examples.
2. Print a compact per-round progress line only at normal verbosity: round, candidates generated/screened/exact, selected distance, score gain, cumulative gain, elapsed time.
3. Write summary fields to the report: requested/effective options, number of accepted moves, evaluated candidates, final radius, stop reason, and timing.
4. Add a developer-only debug option to emit each candidate/commit/rollback topology for fixture diagnosis; keep it off in normal builds.
5. Document that increasing radius can sharply increase work and that SPR is optional refinement, not a universally better replacement for IQ-TREE's established search.

Phase 9 — Validation, regression, and performance

Correctness test matrix

- Move legality: target inside pruned subtree, target equals source, leaf/internal cases, near-root cases, and binary invariants.
- Transactional integrity: topology, edge set, lengths, likelihood, and taxon IDs restored after each rejected candidate.

- Score monotonicity: exact final score after every accepted move.
- Candidate completeness within radius on hand-enumerated small trees.
- Constraints: candidates that violate a constraint are never committed.
- Determinism: same input/options/seed produces same sequence and final tree.
- Checkpoint/resume: interrupted at a committed round boundary produces the same final result as uninterrupted execution.
- Regression: NNI-only runs remain unchanged when SPR options are absent.

Performance protocol

1. Benchmark small, medium, and large alignments under NNI-only, NNI+SPR radius 3, and NNI+SPR radius 5.
2. Record wall and CPU time, peak memory, candidates generated/screened/exact, full cache clears, exact likelihood evaluations, accepted moves, and final likelihood.
3. Plot likelihood gain against added runtime; tune default radius and K from observed cost rather than intuition.
4. Only after baseline profiling, consider screening bounds, partial cache invalidation, candidate sampling, or parallel exact scoring.

6. Proposed work breakdown and deliverables

Milestone	Primary deliverables	Review gate
M1: Baseline	Fixtures, validator, baseline outputs, documented prototype behavior.	Can detect and reproduce mutation failures.
M2: Safe moves	`SPRMove` / `SPRRollback`, apply/rollback APIs, randomized structural tests.	Rollback exactness is proven.
M3: Candidate/scoring	Bounded generator, legality/constraint checks, screening and exact evaluation.	Exact scores and rollback scores agree.
M4: Search integration	IQTree orchestration, score/checkpoint/candidate-tree integration, CLI options.	End-to-end refine mode is stable.
M5: Release quality	Docs, reports, regression/performance results, unsupported-mode guards.	Defaults and supported scope approved.
M6: Extensions	Supertree/partition/mixlen overrides, optional parallelism or advanced heuristics.	Each variant has dedicated tests.

7. Risks and mitigations

Risk	Why it matters	Mitigation
Stale partial likelihoods	Can silently choose wrong topologies.	Begin with full invalidation, exact-score assertions, and only later optimize invalidation.
Incomplete rollback	Corrupts later candidates and makes failures non-local.	Dedicated rollback state; structural snapshots; randomized apply/rollback tests.
Quadratic neighborhood growth	Can make search impractical at scale.	Bounded radius, shortlist K, time/round limits, and measured defaults.
Derived type corruption	Supertrees have linked structures not updated by base rewiring.	Explicit feature guards; dedicated overrides only after design/test work.
Approximate score false positives	Can waste expensive work or cause erroneous commits.	Use approximate scores only for ranking; commit solely on exact re-evaluation.
Regression to established NNI behavior	Core user workflows are sensitive to changes.	Keep feature opt-in; run NNI regression suite with SPR disabled.

8. Suggested initial command-line contract

```
iqtree3 -s alignment.phy --spr-mode refine
iqtree3 -s alignment.phy --spr-mode refine --spr-radius 4 --spr-candidates 16
iqtree3 -s alignment.phy --spr-mode search --spr-radius 3 --spr-rounds 20

# Compatibility alias, if retained:
iqtree3 -s alignment.phy -spr
```

Suggested semantics: `refine` runs only after the ordinary search; `search` remains unavailable or explicitly experimental until integrated into perturbation/stochastic search. Options must be rejected, not ignored, for unsupported tree models or combinations.

9. Immediate next actions

1. Create the baseline SPR fixture set and run the current `-spr` implementation on it.
2. Implement only the structural `applySPR` / `rollbackSPR` transaction and its tests.
3. Review that API before adding any likelihood or search-loop logic.
4. Implement bounded candidate generation and exhaustive small-tree tests.
5. Add conservative exact scoring, then integrate one-move-per-round refinement into `IQTree`.

This document intentionally prioritizes correctness and integration over early speed. In likelihood tree search, a fast topology move that leaves a stale partial likelihood or an incorrect rollback is more damaging than an expensive but verified move.